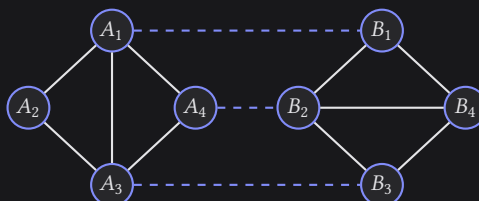


Algorithmique Spécialisée

Recueil d'Exercices

Mots-clés : #Graphes #Kernelisation #FPT



Résumé : Ce document rassemble les énoncés d'exercices et méthodes de résolution pour des problèmes d'algorithmique spécialisée.

Auteur : Ivan Lejeune
Dépôt : github.com/Ivan23BG/HAI904I_Algorithmique_Specialisee
Licence : CC BY-SA 4.0

1 TD1 — Noyaux

Exercice 1.1: Couverture des arêtes par des cliques

On considère le problème suivant, appelé ECC. Étant donné un graphe G et un entier k , le but est de déterminer s'il existe k cliques $C_1, \dots, C_k$ telles que pour chaque arête e du graphe, il existe i tel que $e \in C_i$.

Le but de l'exercice est de montrer l'existence d'un noyau pour ce problème avec au plus 2^k sommets. Pour tout sommet $u \in V(G)$, on dénote $N(u) = \{v, uv \in E(G)\}$ et $N[u] = N(u) \cup \{u\}$. On dit que u et v sont **jumeaux** si $N[u] = N[v]$. On suppose dans tout l'exercice que le graphe n'a pas de sommets isolés.

1. Soit V un ensemble de taille $2^k + 1$ et $C_1, \dots, C_k$ des sous-ensembles de V . Montrer qu'il existe $u, v \in V$ tels que pour tout $i \in \{1, \dots, k\}$, $u \in C_i$ si et seulement si $v \in C_i$.
2. Montrer que si u et v sont jumeaux, alors les instances (G, k) et $(G \setminus u, k)$ sont équivalentes pour le problème ECC.
3. En déduire un algorithme de kernelisation avec au plus 2^k sommets.

Solution

1. On pose $X_v \in \{0, 1\}^k$ le vecteur de taille k défini par $X_v[i] = 1$ si $v \in C_i$ et 0 sinon. Alors, on a 2^k vecteurs différents de taille k et $2^k + 1$ sommets. Par le principe des tiroirs, il existe donc $u, v \in V$ tels que $X_u = X_v$.
2. Soit $C_1, \dots, C_k$ des cliques couvrant les arêtes de G . On pose

$$C'_i = \begin{cases} C_i \setminus \{u\} & \text{si } u \in C_i, \\ C_i & \text{sinon.} \end{cases}$$

Alors, $C'_1, \dots, C'_k$ sont des cliques couvrant les arêtes de $G \setminus u$. Réciproquement, soit $C'_1, \dots, C'_k$ des cliques couvrant les arêtes de $G \setminus u$. On pose

$$C_i = \begin{cases} C'_i \cup \{u\} & \text{si } u \in C'_i, \\ C'_i & \text{sinon.} \end{cases}$$

Alors, $C_1, \dots, C_k$ sont des cliques couvrant les arêtes de G .

3. On peut supprimer les sommets jumeaux du graphe jusqu'à ce qu'il ne reste plus que 2^k sommets. Ainsi, l'instance renvoyé par l'algorithme de kernelisation est équivalente à l'instance initiale et contient au plus 2^k sommets.

Exercice 1.2: Décomposition en couronne : démonstration

Une **décomposition en couronne** d'un graphe G est une partition de ses sommets en trois ensembles C, H, R telle que :

- (i) C est non vide,
- (ii) C est un stable,
- (iii) il n'y a pas d'arêtes entre C et R ,
- (iv) l'ensemble des arêtes entre C et H contient un couplage de taille $|H|$.

1. Montrer que si G a un sommet de degré 1 alors, il admet une décomposition en couronne.

Soit G un graphe avec au moins $3k + 1$ sommets, pour un certain $k \in \mathbb{N}$ et sans sommets isolés. Le but de cet exercice est de montrer que G admet soit un couplage de taille $k + 1$, soit une décomposition en couronne. Soit M un couplage de G maximal pour l'inclusion (c'est-à-dire qu'il n'existe pas de couplage M' de G tel que $M \subset M'$ et $|M| < |M'|$). On dénote $V_M \subset V(G)$ l'ensemble des sommets incidents à une arête de M . On dénote également $I = V(G) \setminus V_M$. On construit un graphe biparti $B = (V_M, I, E')$ où $E' \subset E(G)$ est l'ensemble des arêtes incidentes à un sommet de V_M et un sommet de I .

2. Montrer que I est un stable de G .

Soit $X \subset V(G)$ un vertex cover de taille minimum dans B .

3. Montrer en utilisant le théorème de König que si $X \cap V_M = \emptyset$ alors $|X| = 1$ et il existe un couplage de taille au moins $k + 1$ dans G .
4. On suppose maintenant que $X \cap V_M \neq \emptyset$. On pose $H = X \cap V_M$, $C = I \setminus X$ et $R = V(G) \setminus (X \cup H)$. Montrer que (C, H, R) est une décomposition en couronne de G . (Indication : considérer M' un couplage de taille maximum dans H et appliquer le théorème de König.)
5. Montrer qu'il existe un algorithme en temps polynomial, qui étant donné G calcule soit un couplage de taille au moins $k + 1$, soit une décomposition en couronne.

Solution

Soit G un graphe et x un sommet tel que $d_G(x) = 1$. On note y le voisin de x . On définit les ensembles suivants :

$$C = \{x\}, \quad H = \{y\}, \quad R = G \setminus \{x, y\}.$$

Exercice 1.3: Décomposition en couronne : application au Vertex Cover

En utilisant la décomposition en couronne, construire un noyau pour Vertex Cover avec au plus $3k$ sommets.

Indication : généraliser la réduction vue en cours pour les sommets de degré 1.

Exercice 1.4: Décomposition en couronne : application à SAT

Le but de l'exercice est de construire un noyau linéaire pour SAT basé sur la décomposition en couronne. Étant donné une formule sous forme normale conjonctive et un entier k , le but est de décider s'il existe une affectation satisfaisant au moins k clauses. Soit φ une formule en forme normale conjonctive avec n variables et m clauses.

1. Montrer que si $m \geq 2k$, alors l'instance est positive.
2. Pour tout sous-ensemble X de variables, on note $C(X)$ l'ensemble des clauses dans lesquelles au moins variable de X apparaît. Supposons qu'il existe un sous-ensemble non vide C de clauses tel que :
 - (a) il existe un ensemble X de variables avec $|X| > |C|$ tel que $C(X) = C$ et
 - (b) pour tout $X' \subsetneq X$, on a $|C(X')| \geq |X'|$.

- Montrer en utilisant le théorème de Hall que pour toute affectation qui satisfait un nombre maximal de clauses, toutes les clauses de C sont satisfaites.
3. Montrer grâce au théorème de Hall que tout graphe biparti admet une décomposition en couronne.
 4. Construire un noyau pour SAT avec au plus $2k$ clauses et au plus k variables.

Exercice 1.5: Lemme du tournesol

On appelle d -HITTING le problème suivant. Soit $\mathcal{A}$ une famille d'ensembles sur un univers U telle que chaque ensemble contient au plus d éléments et soit k un paramètre. Le but est de décider s'il existe un ensemble $H \subset U$ de cardinal au plus k tel que pour tout $S \in \mathcal{A}$, on a $S \cap H \neq \emptyset$. Noter que pour $d = 2$, le problème est équivalent à Vertex Cover. Le but de l'exercice est de construire un noyau pour d -HITTING en utilisant un lemme d'Erdős-Rado, aussi appelé le *lemme du tournesol*. Un *tourne-sol à k pétales et de capitule Y* est une collection de k ensembles $S_1, \dots, S_k$ telle que

- i) pour tout $i \neq j$, on a $S_i \cap S_j = Y$,
- ii) les ensembles $S_i \setminus Y$ (appelés pétales) sont non vides.

Lemme 1.1 (du tournesol).

Soit $\mathcal{A}$ une famille d'ensembles sur un univers U , telle que chaque ensemble de $\mathcal{A}$ contient au plus d éléments. Si $|\mathcal{A}| > d!(k-1)^d$ alors $\mathcal{A}$ contient un tournesol à k pétales.

On pourra commencer par traiter la troisième question en admettant les résultats des deux premières.

1. Démontrer le lemme par récurrence sur d .
2. Montrer que le tournesol du Lemme du tournesol peut être calculé en temps polynomial en $|\mathcal{A}|$, $|U|$ et k .
3. En déduire un noyau pour d -HITTING avec au plus $d!k^d$ ensembles.

2 TD2 — Lemme du tournesol

3 TD3 — Compression itérative

Exercice 3.1: Profondeur bornée ne signifie pas FPT

Deux procédures ont une profondeur au plus k . La première crée au plus deux fils par nœud, la seconde au plus n . Le travail local coûte n^2 .

1. Borner leurs temps de calcul et classer ces bornes comme FPT ou XP.
2. Pour une constante c fixée et $k > c$, montrer qu'aucune constante $f(k)$ ne majore n^{k-c} pour tous les entiers n .
3. Un algorithme en temps n^k prouve-t-il que le problème n'est pas FPT?
4. Si le nombre de fils est $k + 1$, la conclusion change-t-elle?

Solution

On note p_1 la procédure qui crée au plus deux fils par nœud et p_2 celle qui en crée au plus n .

1. Le nombre de feuilles pour p_1 est au plus

$$b_1^{d_1}$$

où b_1 est le facteur de branchement et d_1 la profondeur.

Ici, on a $b_1 \leq 2$ et $d_1 \leq k$ donc le nombre de feuilles f_1 respecte

$$\begin{aligned} f_1 &\leq b_1^{d_1} \\ &\leq 2^k \end{aligned}$$

Ainsi, le nombre de nœuds dans l'arbre de recherche pour la procédure p_1 est au plus $2f = 2^{k+1}$. Le temps de calcul en chaque nœud étant exactement n^2 , la procédure p_1 finit en temps au plus $2^{k+1} \cdot n^2$.

En posant $f(k) = 2^{k+1}$, on obtient que p_1 termine en $f(k)n^2$ donc la procédure p_1 est FPT.

On procède de même pour p_2 :

$$b_2^{d_2}$$

avec $b_2 = n$ et $d_2 = k$.

On a alors un temps de calcul en au plus $2n^k \cdot n^2 = 2n^{k+2}$. Le facteur k étant dans l'exposant de n , la procédure p_2 est XP.

2. À k fixé, $f(k)$ est une constante. Or n^{k-c} est une fonction avec $\lim_{n \rightarrow \infty} n^{k-c} = +\infty$. Donc aucune constante ne majore cette fonction pour tout n .
3. Un tel algorithme ne suffit pas à prouver qu'un problème n'est pas FPT, pour cela, il faut prouver qu'il est W[1] difficile.
4. Si le nombre de fils à chaque nœud devient $k + 1$, le nombre de feuilles devient 2^{k+1} qui est toujours $O(2^k)$. La procédure est donc toujours FPT.

Exercice 3.2: Closest String

On reprend les mots $s_1, \dots, s_r \in \Sigma^L$, le rayon d et le pseudo-code du cours. La distance de Hamming $d_H(x, y)$ est le nombre de positions où x et y diffèrent. L'appel initial se fait avec $(z, l) = (s_1, d)$.

1. Soit c une solution et s_i un mot avec $d_H(z, s_i) > d$. Dans un ensemble P de $d+1$ positions de désaccord, montrer qu'il existe $p \in P$ tel que $c[p] = s_i[p] \neq z[p]$. Pourquoi ne suffit-il pas de dire seulement que c et z diffèrent quelque part dans P ?
2. Montrer que, si $d_H(z, c) \leq l$, au moins un appel enfant vérifie $d_H(z^{(p)}, c) \leq l-1$. Justifier l'invariant à la racine.
3. En déduire que l'algorithme trouve une solution si elle existe et n'en renvoie jamais une fausse. Traiter le cas $l = 0$.
4. Pour $d \geq 1$, borner le nombre de nœuds et obtenir un temps $O((d+1)^d rL)$. Traiter séparément $d = 0$.

Solution

1. Si c est une solution alors $d_H(c, s_i) \leq d$. Ainsi, pour $d+1$ positions de désaccord entre s_i et z , il y en a au moins une qui vérifie $c[p] = s_i[p]$ sinon $d_H(c, s_i) > d$ ce qui contredit le fait que c est une solution.
Si on a seulement que c et z diffèrent alors rien ne permet de garantir que la solution est à bonne distance de tous les s_i .
2. Supposons que $d_H(z, c) \leq l$. Chaque appel enfant est fait en modifiant une des positions de désaccord parmi $d+1$ positions. Par la question précédente, au moins une de ces positions correspond à un désaccord entre z et c . Ainsi, l'appel enfant qui modifie cette position en le caractère correspondant dans c vérifie bien $d_H(z^{(p)}, c) \leq l-1$.
Cette preuve étant vraie à chaque étape, elle est en particulier vraie à la racine de l'arbre.
3. L'algorithme réduit d strictement à chaque étape dans une feuille de l'arbre, ainsi la solution se trouve en descendant un nombre fini de fois dans l'arbre. Si une solution existe, l'algorithme la trouve, car il va explorer toutes les possibilités qui y mènent. L'algorithme renvoie une solution après avoir vérifié que $\forall i, d_H(z, s_i) \leq d$, ainsi, il ne peut jamais renvoyer une solution fausse.
Si $l = 0$ alors tous les s_i doivent être identiques et correspondent à la solution, sinon il n'en existe pas.
4. A chaque étape de l'arbre, il y a au plus $d+1$ positions de désaccord sur lesquelles on va brancher. On va descendre dans l'arbre au plus d fois pour atteindre la solution, ce qui fait $(d+1)^d$ feuilles.
Le cout local en chaque nœud est le temps de parcours d'un mot donné, ce qui se fait en $O(L)$ et le parcours des mots pour en trouver un qui est à distance $\geq d+1$, ce qui se fait en $O(r)$. Le nombre de nœuds dans l'arbre est au plus $2(d+1)^d$, ce qui fait un temps de calcul total de $O((d+1)^d rL)$.

Exercice 3.3: Suppression de cycles dans un tournoi

Un **tournoi** est un graphe orienté sans boucle tel que : pour tous sommets distincts u, v , exactement un des arcs $u \rightarrow v$ ou $v \rightarrow u$ existe. Étant donné un tournoi T et un entier $k \geq 0$, on cherche au plus k sommets dont la suppression rend T acyclique.

1. Montrer qu'un tournoi contient un cycle orienté si et seulement s'il contient un triangle orienté.
Indication : considérer un cycle orienté de longueur minimum et l'arc entre son premier et son troisième sommet.
2. Écrire un algorithme FPT pour ce problème, démontrer sa correction et analyser sa

complexité.

Solution

1. Soit $C = c_1, \dots, c_n$ un cycle orienté minimal de longueur $n \geq 4$ dans un tournoi. Comme C est dans un tournoi, l'arc $c_1 c_3$ existe. Si cet arc est $c_3 \rightarrow c_1$ alors c_1, c_2, c_3 forme un cycle de longueur $3 < n$. Si cet arc est $c_1 \rightarrow c_3$ alors $c_1, c_3, \dots, c_n$ forme un cycle de longueur $n - 1 < n$.
Ainsi, $n \leq 3$. Comme un cycle de longueur 2 n'existe pas dans un tournoi (car il n'y a qu'un arc entre toute paire de sommets), on a $n = 3$. Donc tout cycle orienté implique l'existence d'un triangle orienté (la réciproque est évidente).
2. D'après la question 1, un tournoi est acyclique si et seulement s'il ne contient aucun triangle orienté. Pour détruire tous les cycles avec au plus k sommets, il suffit de trouver un triangle orienté (u, v, w) et de brancher sur la suppression de u , de v ou de w .

TOURNAMENTFVS

Algorithm 1

Input : Tournoi $T = (V, A)$, entier $k \geq 0$

Output : Vrai si T peut être rendu acyclique en supprimant $\leq k$ sommets, Faux sinon

if T ne contient aucun triangle orienté **then return** Vrai

if $k = 0$ **then return** Faux

Trouver un triangle orienté (u, v, w) dans T

// Temps polynomial $O(n^3)$

for chaque $x \in \{u, v, w\}$ **do**

if TOURNAMENTFVS($T \setminus \{x\}, k - 1$) **then**

return Vrai

end

end

return Faux

Démonstration de correction :

- *Exhaustivité* : Soit $S \subseteq V$ une solution de taille $|S| \leq k$. Si T contient un triangle orienté $\{u, v, w\}$, S doit obligatoirement contenir au moins un sommet de ce triangle (sinon le triangle survit dans $T \setminus S$, ce qui contredirait l'acyclicité). Explorer les trois branches $T \setminus \{u\}$, $T \setminus \{v\}$ et $T \setminus \{w\}$ garantit de préserver l'existence d'une solution valide si elle existe.
- *Conditions d'arrêt* : Si aucun triangle n'existe, T est acyclique (question 1), la branche est valide. Si $k = 0$ alors qu'il reste au moins un triangle, le quota est épuisé et la branche échoue.

Analyse de complexité :

- Trouver un triangle orienté s'effectue en $O(n^3)$ en testant tous les triplets.
- L'arbre de recherche possède un facteur de branchement au plus 3 et une profondeur maximale de k (puisque k décroît de 1 à chaque niveau).
- Le nombre de nœuds dans l'arbre de recherche est donc majoré par $O(3^k)$.

La complexité globale est $O(3^k \cdot n^3)$, ce qui est bien un temps FPT par rapport au paramètre k .

Exercice 3.4: SAT avec peu de variables vraies

Soit une formule CNF φ dont chaque clause contient au plus d littéraux avec $d \geq 2$ fixé. Un littéral est une variable x ou sa négation $\neg x$. On cherche une affectation satisfaisant toutes les clauses et rendant vraies au plus k variables. Le paramètre est k .

1. Écrire un algorithme basé sur une stratégie de branchement, démontrer sa correction et analyser sa complexité.
2. Expliquer le rôle de l'hypothèse « d fixé » pour obtenir un algorithme FPT de paramètre k . Si d fait désormais partie de l'entrée, l'algorithme est-il FPT pour le paramètre $k + d$?

4 TD1 — Noyaux